

ChArm Proposal

Making Intelligent Things - CS358(b)

Allan Burnier, Sipke Fokkema, Tamara Horak, Nika Horvat, Fazil Souissi, Haobin Wang, Jiayi
Zhang

Introduction.....	3
Main parts of the scara chess build.....	3
Detailed Description.....	4
Mechanical.....	4
Software.....	8
Electronics.....	9
Milestones.....	11
Milestone 1.....	11
Milestone 2.....	11
Milestone 3.....	11
User Stories.....	12
Known and Useful Resources.....	13
Bill of Materials.....	14
Risk Assessment and Potential Challenges.....	15
Mechanical.....	15
Electronics.....	15
Software.....	16
Camera Distance and Field of View.....	17
Image Capture Latency (Lag).....	17
Organization.....	18
Appendix.....	20
Summary of risks table.....	20

Introduction

Our project aims to design and build a SCARA robotic arm capable of autonomously moving chess pieces on a board. By integrating computer vision for board state recognition and a chess engine for decision-making, the system can play complete games against a human opponent. This combination of precise mechanical control and intelligent software creates an interactive and automated chess-playing platform.

Main parts of the scara chess build

Chessboard and pieces

A custom-designed chessboard along with all 32 chess pieces, optimized for reliable detection and manipulation by the robotic system.

SCARA arm base

A stable enclosure that houses the mechanical structure, electronics, and control components of the SCARA arm.

SCARA Arm

The primary robotic assembly responsible for precise movement and positioning, enabling accurate manipulation of chess pieces across the board.

Camera Arm

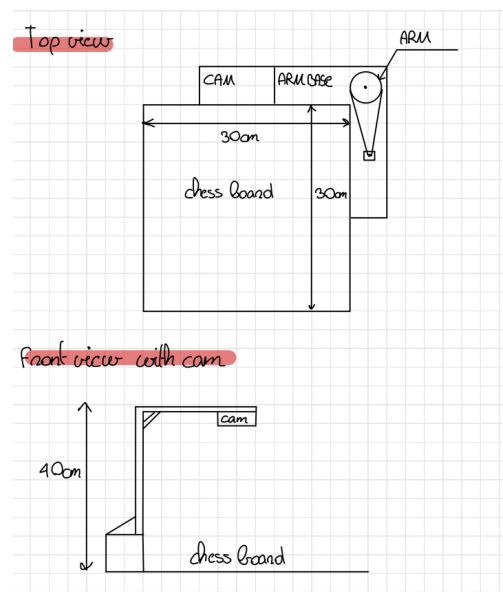
A dedicated support structure that positions a camera above the chessboard, allowing for real-time computer vision and board state recognition.

Detailed Description

Mechanical

The main arm assembly will consist of 4 main parts.

Here is a sketch to see what the position of each element will be and how it will all get together with the chess board in place



The arm base

The arm base will be a box laser cutted out of MDF. The only constraint on it is its position since to cover the maximum range it needs to be in one of the angles of the chessboard.

The base will have to serve multiple purposes.

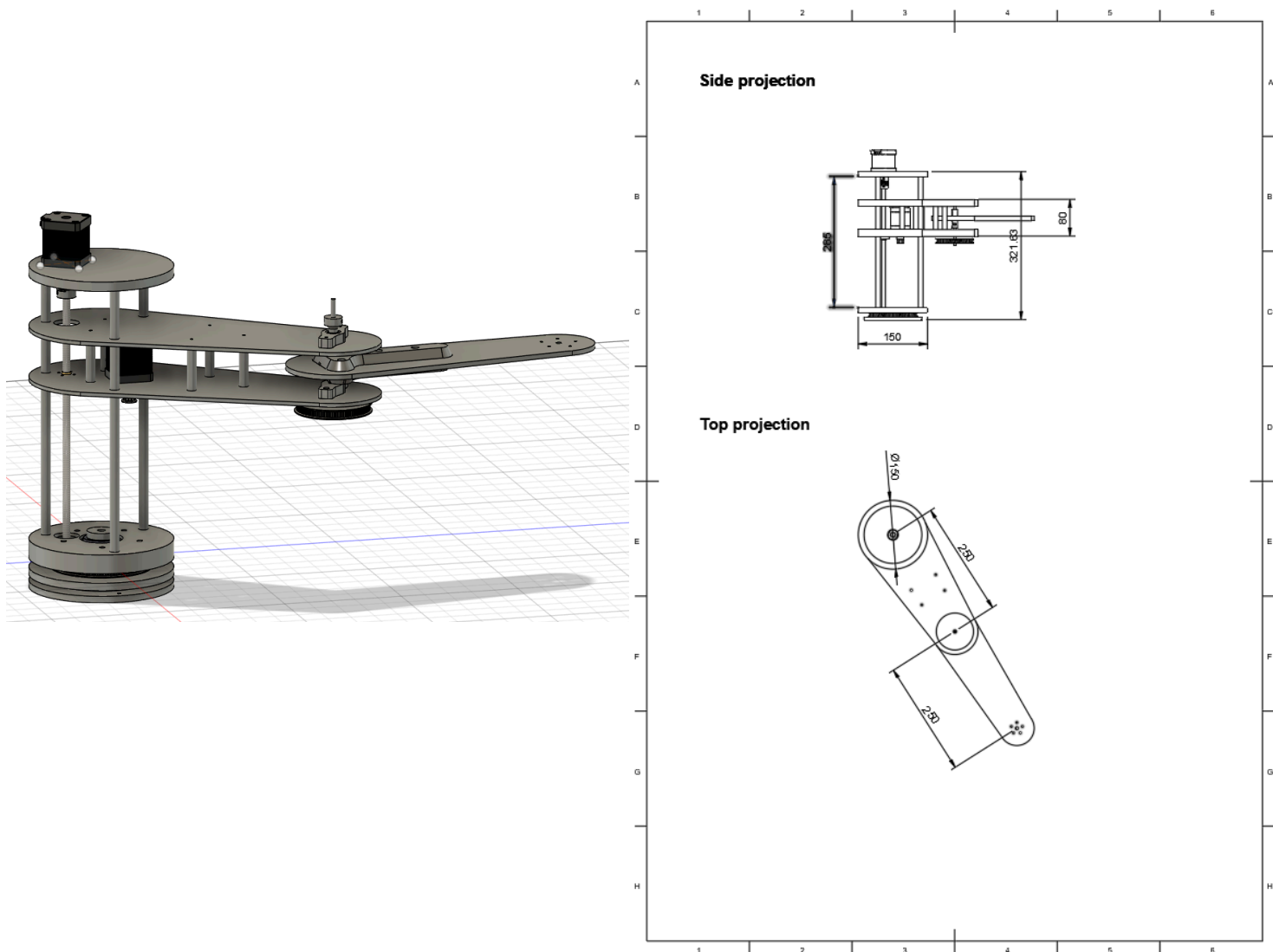
- **provide stability** Since our arm will be ~50cm long it has a potential to fall over. So the base will need to be heavy enough.
- **Cable management** the box will have an opening on one of the sides to allow routing cables, motor drivers, the arduino and different cables needed.
- **Screen mount** our build will also have a small lcd screen and a few buttons to interact with the robot arm. The Screen will be mounted on that base.

The camera arm

A minimalist structure to hold the camera on top of the chess board. It will have to be taller than the robot arm so it doesn't get into the way of the SCARA arm. It will be made out of MDF and have a hanger like structure with 3D printed links.

The SCARA arm

The arm is the main assembly of our project. It will have three degrees of freedom.



Details of the main arm + forearm

- **The main arm**

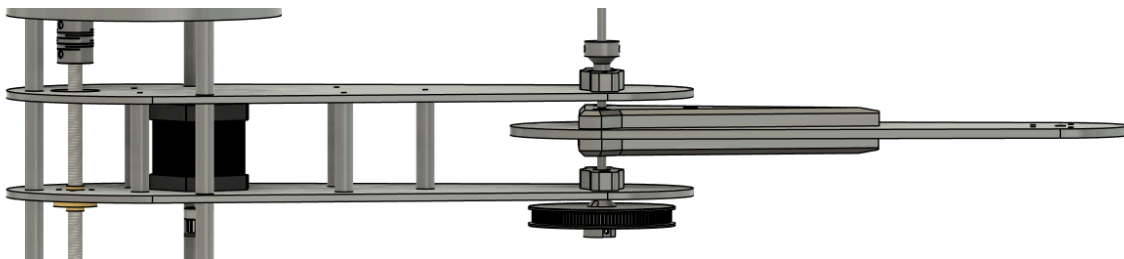
will be composed of two platforms. Both platforms will be cut out of MDF and held together with 3d printed pillars that will screw on each side with m3 screws.

- **Top main arm platform**

- 4mm MDF

- **Bottom main arm platform**

- 4mm MDF plates glued together with holes for the t8 lead nut and the nema 17 motors



- **The forearm**

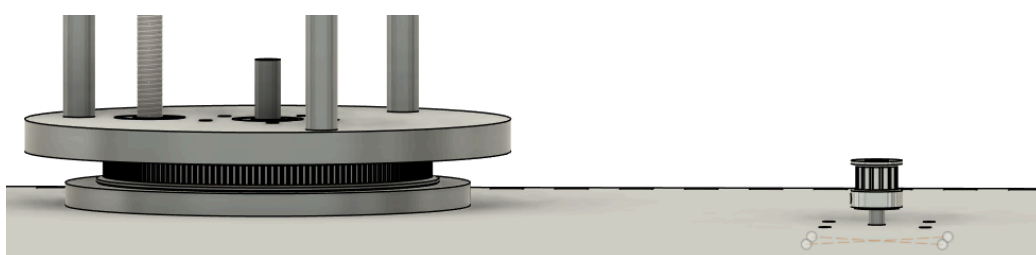
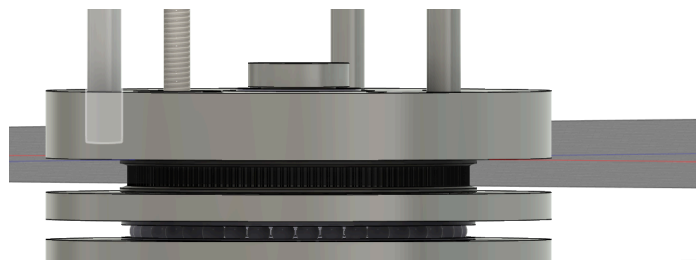
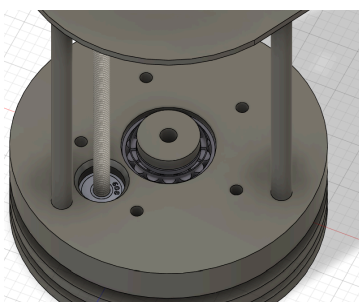
It will only be a single plate of 4mm thick MDF. It will have a 3d printed clamping mechanism attached on the bottom and the top part it will allow for the forearm to be clamped tight on the 5mm axle. The forearm will also have 4 m3 screw holes to have the gripper attached to it.

Different degree of freedom and linked mechanical parts

Rotation of the arm.

The rotation is induced by a nema 17 motor that will stay in the arm base.

- To ensure smooth gliding and side forces.
- We will use a flat axial bearing at the bottom so the whole assembly weight will sit on top of it

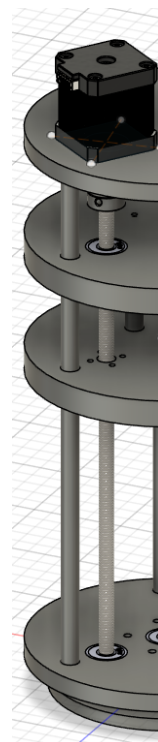
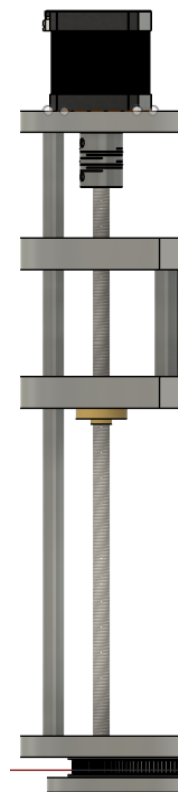


- and a conical bearing on top
- Both bearing will be tightened together using a m8 screw

Up down motion of the arm :

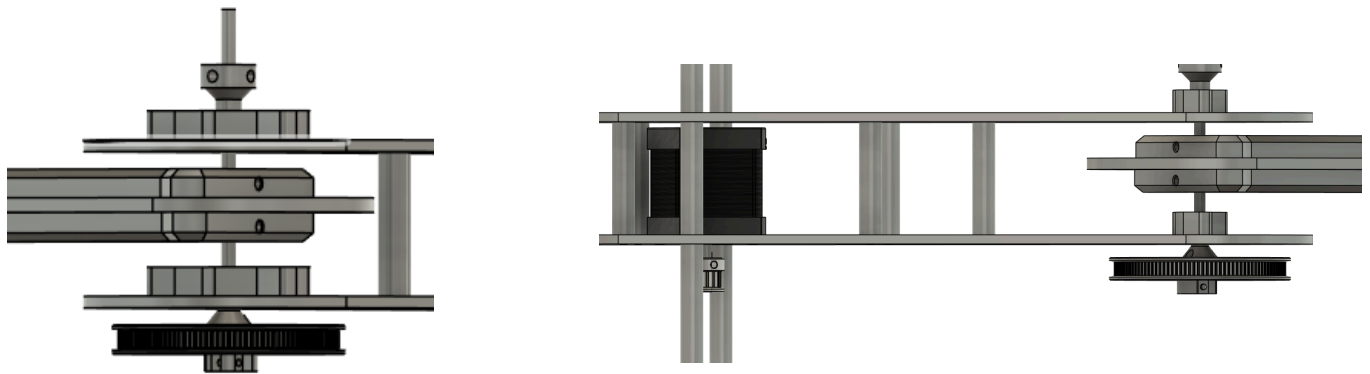
For the main arm to slide up and down a few components are involved in this motion

- **The rails** There will be 3 main slider rails that will resist any rotational movement of the main arm and provide some rigidity to the structure. For the rails we choose to go with some aluminium rods to stay as light as possible. The rails will be held to the base and to the top platform by clamping.
- **Lead screw + lead nut + nema 17** The upward and downward motion of the main arm will be produced by a nema 17 stepper coupled to a t8 lead screw. The lead nut will be attached to the main arm. Note that the lead screw will be supported by 2 608 bearings, one at the base, and on the upper platform of the main arm. The nema 17 will sit on the top platform and be directly coupled with a m8 to m5 shaft coupler.



Forearm joint rotation :

The forearm rotation is induced in the same way as the rotation of the base. By a nema 17 that sits directly on the main arm it will drive a pulley connected to another bigger pulley which is also connected to a m5 shaft where the forearm is clamped to it. At the joint there are 2 625 bearings to support the m5 shaft for it to rotate smoothly. We decided to go with smaller bearings to reduce the weight.



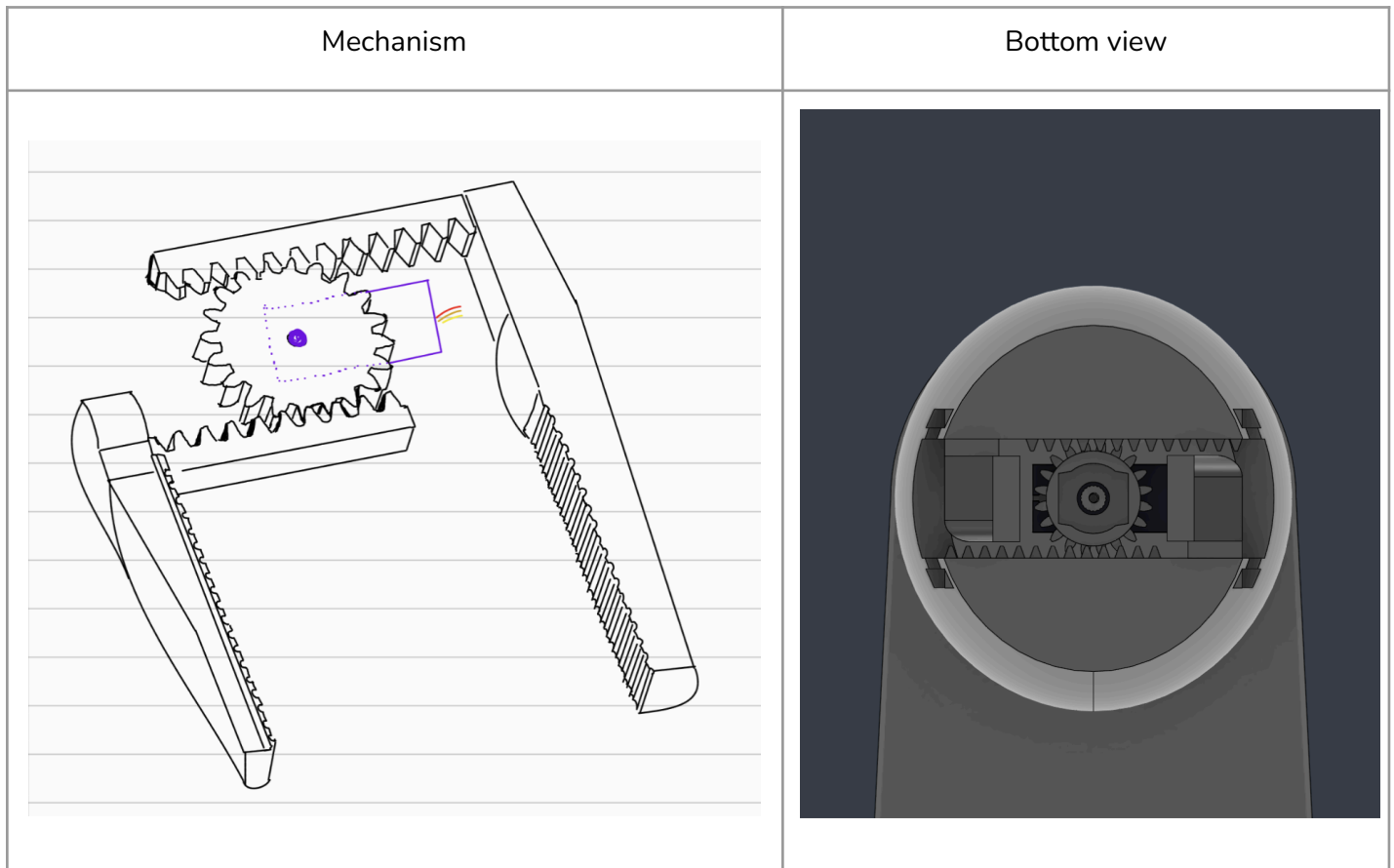
The Gripper

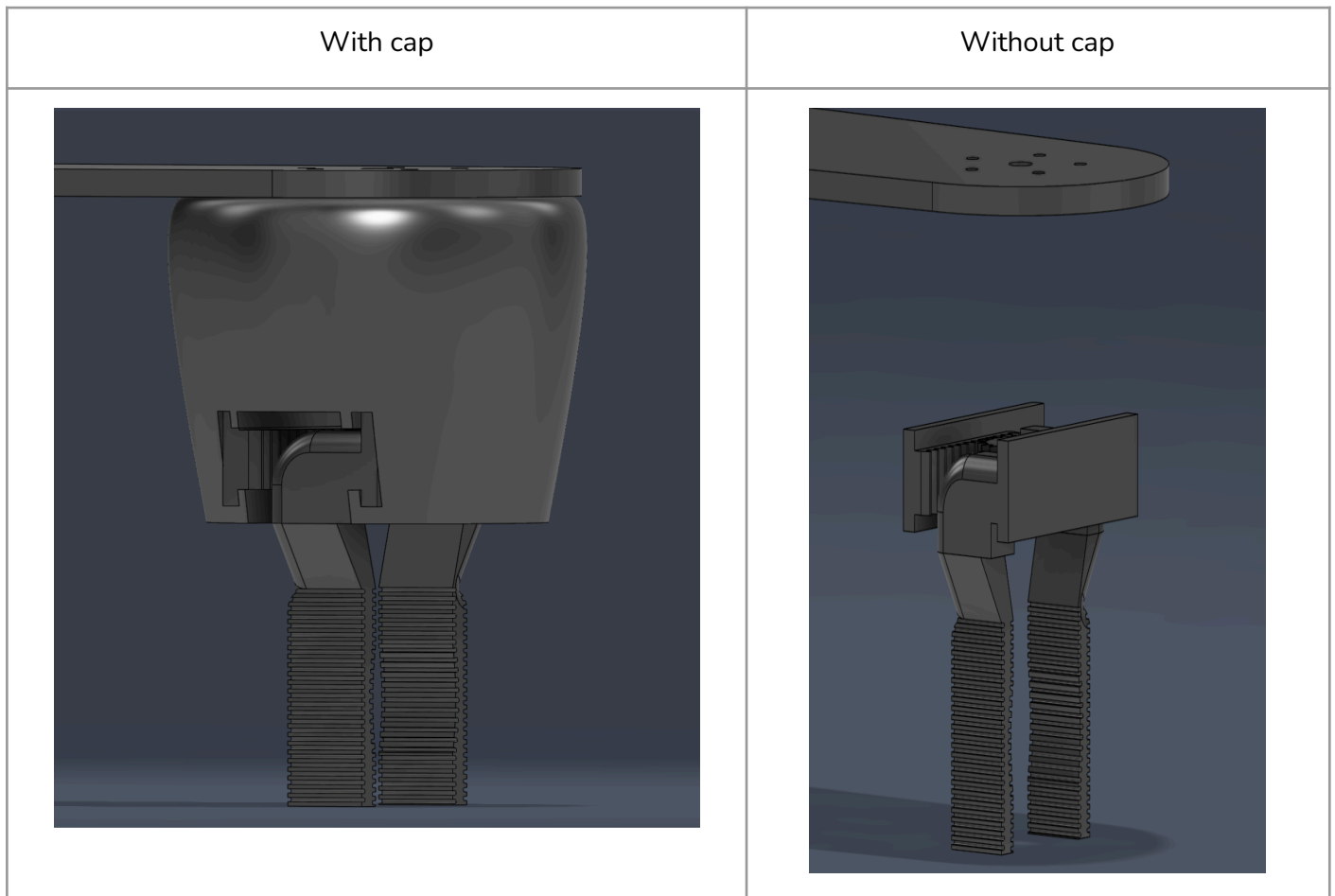
Two gripper concepts were explored before settling on the current design. The first used 4 gears driving two opposing arms, and the second used 3 gears with a worm screw and 3 arms. Both relied on a rotational closing motion, which turned out to be the core problem: as the arms close by rotating inward, they sweep sideways and risk disturbing neighboring pawns on the chessboard. This made both concepts unsuitable regardless of how well the transmission worked.

The current design addresses this directly. It uses a single gear with a rack-and-pinion mechanism to drive two jaws in a purely linear clamping motion, eliminating any lateral sweep. This is both mechanically simpler and much better suited to the tight spatial constraints of the chessboard.

The outer housing was modeled using a loft, and the jaw geometry was refined using a loft with guide rails to improve shape continuity beyond an initial block-based draft.

Mechanism :





The current validated sizing is a 0.5 mm wall thickness for the gripper. The next step is to compute the gear ratio to determine the correct gear and pinion diameters, which will allow the rack geometry, jaw travel, and clearances to be finalized. The plier size (currently 15.8 mm) is also being reassessed to ensure sufficient gripping capacity across the different chess piece geometries.

Print of the first prototype :



Software

Our primary objective for Milestone 1 is to hardcode the robotic arm's movements to ensure it can reliably grab and relocate objects. By implementing Inverse Kinematics, we will compute the required joint angles and linear actuator positions for a given XY target and height for the arm to follow the most efficient path to any target on the board.

For Milestone 2, the focus shifts to vision and calibration. We will identify square occupancy, piece color, and the board's boundaries through manual camera calibration to help accurately recognise each square, ensuring the movement detection and robot positioning is more precise. Using OpenCV, we will use HSV Color Space conversion to detect pieces. Unlike standard RGB, HSV is highly resistant to shadows, and allows us to count "black" or "white" pixels in each square to detect the color of pieces. To maximize contrast, we will use a rose and green chessboard, which ensures that the black and white pieces stand out clearly against the background during HSV filtering.

Milestone 3 represents the "intelligence" phase of the system. The program will detect movement by analyzing the difference in square occupancy between two distinct states. This will be managed by a physical button: the user presses it before and after their turn to capture the current frame of the board, allowing the system to identify the move made. We will integrate the Stockfish engine to analyze the position and generate the robot's counter-moves. To manage this data, we will use the python-chess library to store game states, validate move legality, and maintain a complete history of the match. Finally, when the robot discards an opponent's piece, the robot will discard it in a designated area coded in the program. The main program will have the animated chessboard with all the pieces of the last captured frame with all the pieces on the correct space of the captured frame, the main will also have displayed the camera feed with along with the grid lines for each square.

Electronics

Power distribution

A 12V, 5A DC power supply is needed to power the stepper motors. Starting from the power supply, it is connected to the CNC Shield and the buck converter in series. All components share a common ground.

The CNC Shield provides 12V to the three NEMA 17 stepper-motors through the 3 A4988 motor drivers. This has a peak draw of 2.5A.

The buck converter converts the 12V to 5V to power the ESP32-Cam (500mA when using wifi or flash), the 1 SG90 Servo (650mA), The Adafruit RGB LCD Shield Rev B1 (100mA)

and the Arduino Mega 2560 Rev3 (100mA) in parallel. The circuit has a potential 1.35A peak current draw, potentially more.

Arm motors

Our SCARA Robot arm has 3 axis of movement:

- Z axis, moving the arm up and down on the pillar
- First circular movement around the pillar
- 2nd circular movement at the end of the first arm.

These axes are made possible by 3 NEMA 17 stepper motors connected through 4988 motor drivers. The Arduino Mega controls the steppers via STEP and DIR signals to the CNC Shield (pins 22-27).

We add 2 limit switches per axis to detect whether we hit the minimum or maximum limit. We can use the provided limit switch pins on the CNC shield that map back to pins 9, 10 and 11 on the Arduino mega.

Grabber servo

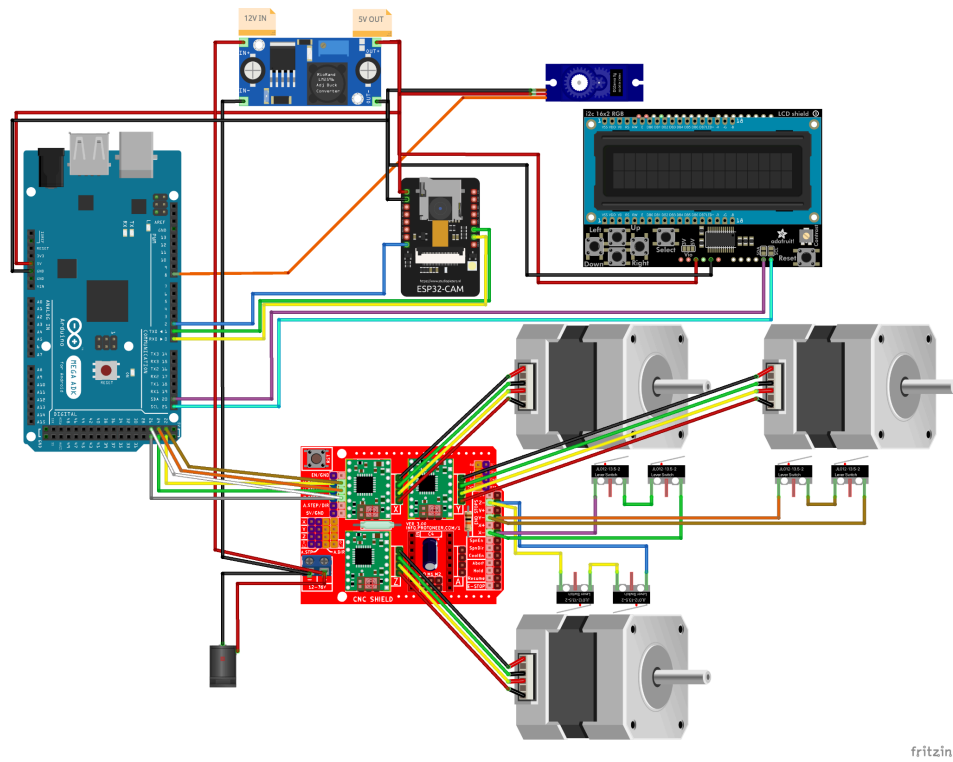
We use a SG90 servo motor to control the closing and opening of our arm's hand. It is powered after the buck converter with 5V and controlled with the Arduino Mega via PWM signal (pin 8).

Vision

To see and analyse the chess board, we use an ESP32-Cam positioned above the chess board. It is powered with 5V after the buck converter and sends signals to the Arduino Mega via its GPIO 1 and GPIO 3 serial pins (TX and RX, respectively) to the Arduino's RX0 and TX0 pins (pins 0 and 1). The built-in LED/Flash is connected via GPIO 4 to pin 2 on the arduino.

Inputs

To input settings and tell the arm it is its turn, we use an Adafruit RGB LCD Shield Rev B1. It is powered with 5V after the buck converter and sends signals to the Arduino Mega's SDA and SCL pins (pins 20 and 21).



Electronic diagram

Milestones

Milestone 1

The objective of Milestone 1 is to develop a functional SCARA robotic arm capable of autonomously picking up and relocating chess pieces without relying on computer vision. At this stage, the focus is on the mechanical assembly and low-level control of the robot, the arm must be able to execute reliable movements, actuate the gripper, and place an object at a specified target location. The motion will initially be hardcoded, and inverse kinematics will be implemented to compute the required joint rotations and vertical displacement for a given board position. This milestone validates that the robot can physically perform the core pick-and-place task with sufficient precision and repeatability.

Milestone 2

The goal of Milestone 2 is to equip the system with the ability to perceive and interpret the chessboard state, prototype the chess board and the pieces. This includes integrating the overhead camera, calibrating its position with respect to the board, and detecting the limits of the board as well as the occupancy of each square. The system will also distinguish between black and white pieces using image processing techniques in OpenCV, with HSV colour space used to improve robustness against lighting variations and shadows. At the end of this milestone, the robot should be able to identify whether a specific square is occupied, determine the colour of the occupying piece, and accurately map visual information to physical board coordinates.

Milestone 3

Milestone 3 represents the intelligent layer of the project, where the system becomes capable of playing a full chess game against a human user. The program will detect the player's move by comparing two captured board states, taken before and after the user's turn, for example through a button-triggered update. The recognised move will then be stored and validated using a chess-state management library such as python-chess, while Stockfish will be used to compute the robot's counter-move. The robotic arm will then execute this move physically on the board, including moving captured pieces to a designated discard area. By the end of this milestone, the full pipeline, from board-state recognition to chess-engine decision-making and physical move execution, will be operational.

User Stories

1. CS-358 professor / future student teams

One important user group is the CS-358 professor and future student teams. For them, the main value of the project is not only the final demonstration but also its usefulness as a reusable learning platform.

Our SCARA arm should therefore be more than a one-time prototype. It should be documented and structured clearly enough that future teams can understand, reproduce, and improve it. Since we would like the project to move toward a chess-playing robotic system, it is important that the platform remains modular and easy to extend.

To support this, the project should provide a clear GitHub repository with the control code, CAD files, wiring diagrams, concise setup instructions and a simple Bill of Materials (BOM). The project will be especially valuable if future teams can build on it to improve motion control, piece handling, or chess-related interaction.

2. Chess player / interactive user

Another key user group is the chess player interacting with the robot. For this user, the most important aspect is that the interaction feels clear, accurate, and engaging.

The user should be able to make a move on the board and easily understand how the robotic arm responds. Even in an early version, the system should already demonstrate the idea of a robot physically participating in a chess game.

This means the arm should move pieces in a controlled, visible, and safe way, while making the turn-taking process easy to follow. The project succeeds for this user if the interaction feels intuitive and the robot is perceived as an active part of the game rather than just a mechanical demo.

3. EPFL IC faculty / outreach staff

A third important user group is the EPFL IC faculty and outreach staff, who may use the project during presentations or public events. For them, the priority is reliability, safety, and ease of use.

From this perspective, the SCARA arm should be easy to set up, simple to operate, and robust enough to run repeated demonstrations without frequent intervention. A

chess-related demo is especially suitable because it is visually clear and easy for visitors to understand.

The project will be successful for this user group if it can be set up quickly, run reliably in public, and serve as an engaging example of robotics at EPFL.

Known and Useful Resources

- The electronic components choices and electronic circuit was inspired by the 2025 Container Terminal 1 project done previously in CS-358. URL : <https://github.com/epfl-cs358/2025fa-terminal1>
- The software part of the project was mostly inspired by the "Chess Playing Robot Arm That Will Beat You!" from hackster.io, and also by the YouTube video: Building a Chess Robot Using the AR4 MK3 and Python. In particular, this is where we found the idea of capturing the current frame of the board and using the Stockfish library. URL: <https://www.hackster.io/ricpd/chess-playing-robot-arm-that-will-beat-you-678035>, <https://www.youtube.com/watch?v=qwZDZwg1g9o>
- The library we will use for the computer vision part is OpenCV (<https://opencv.org>).
- The chess engine will be provided by Stockfish (<https://stockfishchess.org>).
- To store game states, validate move legality, and maintain a complete history of the match, we will use python-chess (<https://python-chess.readthedocs.io/en/latest/>)

Bill of Materials

Part Name	Number Part	Part Price	Total Price	Link	Justification
Tapered roller bearing LM11949	1	1	5	view	conical bearing used for the base rotation
Axial ball bearing 51116	1	1	9	view	axial bearing used for the base rotation
nema 17 pulley (GT2)	2	4	8	view	used on the motor shaft for the base and the arm joint
608 bearing	2	1	2	mechanical workshop	t8 lead screw support
625 bearing	2	1	2	view	for arm joint rotation
GT2 closed timing belt 640mm	2	6,9	13,8	view	
lead screw t8x2	1	21	21	view	for upward and downward movement of the arm
lead screw t8 nut (2mm pitch)	1	1	1	view	link t8 screw
5mm to 8mm coupler	1	4	4	view	link t8 screw to nema 17 motor shaft
Aluminium rod	1	5	5	mechanical workshop	sliding rails for the arm
NEMA 17 (5 CM one)	3	10	30	view	used for different rotations/movement of the arm
Connector Cables for the NEMA 17 Steppers	3	2,9	8,7	view	to connect the motors to the circuit
A4988 driver	3	3,9	11,7	view	to drive the nema 17 motors
CNC shield	1	3	3	view	to drive the 3 axis movement
AIThinker ESP32-CAM	1	4	4	course stock	camera to implement computer vision
Arduino Mega 2560	1	14	14	course stock	microprocessor signaling all components of the circuit
Servo MG90S	1	7,9	7,9	view	motor for closing and opening of the gripper
Adafruit RGB LCD Shield Rev B1	1	20,14	20,14	view	display and buttons to interact with the arm

LM2596 buck converter	1	6,95	6,95	view	to convert 12V to 5V to power the components
DC 12V, 5A power supply	1	9,67	9,67	view	to power the whole arm
DC Barrel Power Jack	1	1,18	1,18	view	to connect the power supply to the circuit
Gain cable management	1	2,3	2,3	view	cable management
KW11-3Z limit switches	6	1,295	8	view	may be used to restrict the different movement
Total			176,34		

Risk Assessment and Potential Challenges

Mechanical

ARM rigidity

- Issue: The arm structure may have too much flex and so not as precise as wanted.
- Mitigation:
 - Add rigidity pillars and attachment that go the whole way of the arm / forearm
 - Have a very light chess set and gripping system

Up and down motion stutter

- Issue: the arm sliding on the 3 guide rails may result in friction that will cause some unwanted movements
- Mitigation:
 - Add some “pallier lisse”(available in the mechanical workshop) to the MDF structure so it will slide better

Arm reach

- Issue: even though calculated the arm may struggle to precisely reach some chess pieces
- Mitigation:
 - Adapt the chess board size and placement

Gripper

- Issue: The gripper may fail to generate enough force to securely hold and lift some chess pieces.
- Mitigation:
 - Add silicone or rubber pads to the inner jaw surfaces to increase friction.
 - Size the pinion radius and jaw geometry so that the required gripping force remains below the servo torque limit with a safety factor.
 - Test the gripper using the heaviest chess piece early in prototyping.
- Issue: A piece may initially be gripped correctly but slip while the arm is moving.
- Mitigation:
 - Use smoother motion profiles during lifting, transport and testing.
 - Limit acceleration while carrying a piece.

Electronics

ESP32-CAM Power Instability

- Issue: The ESP32-CAM draws short bursts of high current when the camera or Wi-Fi module is active, which can reset the board if the 5 V rail isn't stable.
- Mitigation:
 - Keep the 5 V line short and thick.
 - Use a low-ESR capacitor ($\geq 470 \mu\text{F}$) across the ESP32's 5 V and GND pins.
 - Test Wi-Fi connection while motors are running to check for power dips.

Insufficient current to motors

- Issue: Moving the arm using all three stepper motors at the same time might draw too much power and prevent the motors from moving correctly
- Mitigation
 - Testing how many motors can run at once
 - Making it so only one motor can move at a time

Buck converter bottleneck

- Issue: the 5 components after the buck converter have a peak current draw of 1.35A, potentially more. The buck converter could overheat.
- Mitigation
 - Monitoring and testing the circuit for power spikes
 - Attaching a heat sink to the buck converter

Software

ESP32-CAM Resolution Quality

- Issue: We plan to use the camera to keep track of chess pieces to give instruction to which one the robotic arm needs to move. Low camera quality can make identification considerably harder
- Mitigation:
 - Using high contrast colors between chess pieces and the chess board (making the chess board green and pink).
 - Test using different lighting conditions or positions of the camera.
 - We could use the camera's flash when capturing each frame to improve image quality and visibility.
 - As a last resort, we can add a support to hold a phone above the board, using its flash to provide additional lighting, or even use the phone's camera.

Special Rule Cases

- Issue: Handling castling moves can be challenging, as this is the only case where the robot does not perform a standard single-piece move. Additionally, the resulting board changes may appear unusual to the engine compared to typical moves.
- Mitigation:
 - We will handle castling operations separately when the engine instructs the robot to perform such a move.
 - Detection of castling moves performed by the opponent will also be treated as a special case and handled independently.

Low Color Saturation

- Issue: The ESP32-CAM aggressively normalizes colors under indoor lighting, washing out colors into near-grey. This can make HSV detection unreliable.
- Mitigation:
 - Disable the ESP32-CAM's Auto White Balance (AWB) in the Arduino firmware to prevent the camera from automatically normalizing colors, which would preserve the original saturation of the squares
 - Use more saturated colors for the board squares
 - Artificially boost the saturation channel in OpenCV before applying HSV thresholding

Camera Distance and Field of View

- Issue: At the required 40cm mounting height for the robot arm, the printed squares only cover a small portion of the camera's field of view, making the individual squares too small for reliable color detection. Additionally, The camera occasionally detects background noise, shadows, or objects outside the target area, leading to incorrect identification.
- Mitigation:
 - Zoom lens to fill the frame better at 40cm.
 - Use Area Filtering to ignore detected shapes that are too small or too large to be valid pieces.

Image Capture Latency (Lag)

- Issue: The ESP32-CAM has a transmission delay where the captured image is sometimes a "stale frame" from a previous cycle.
- Mitigation:
 - Add a buffer flushing routine in the code to ensure the system always processes the most recent frame.

Organization

Phase	Tasks	30 - 5 Apr. Week 7	6 - 12 Apr. Break	13 - 19 Apr. Week 8	20 - 26 Apr. Week 9	27 - 3 May Week 10	4 - 10 May Week 11	11 - 17 May Week 12	18 - 24 May Week 13	25 - 31 May Week 14
Cutting and printing the parts and assembly	- Cutting MDF parts - Printing 3D parts - Building the arm - Building the grabber - Adding the electrical circuit									
Finalizing Milestone 1	- Hardcoding movement - Testing the grabbing and movement									
Milestone 2	- Adding OpenCV Vision - Calibrating the movement with the vision - Making the Chess board									
Milestone 3	- Adding Stockfish integration - Adding buttons and screen interface									

Making and assembling the parts

This phase is about making or acquiring all the necessary parts for the project build. This entails laser cutting all parts from MDF (arm, box, ...) and 3D printing the grabber and

gears. Next, a first version of the arm will be assembled with the electrical so we design the software for it. We will be coming back and modifying parts throughout the project when necessary.

Responsibilities:

Mechanical build : Allan

- CAD of the whole arm assembly
- Prototyping of the arm assembly

Gripper : Fazil, Nika

- Design chess pieces
- Continue working on the CAD of the new gripper
- Adjust the dimensions of the gripper and gears accordingly

Electrical : Sipke

- Assemble the electrical circuit
- Test the currents
- Sketch out the future I/O for the software

Finalizing Milestone 1

While the physical part of milestone 1 is being completed, part of the team can write the software. This means hardcoding the movement of the arm and grabber. When both the mechanical and software parts are put together, we can test whether the arm works as intended.

Responsibilities

Software : Tamara, Haobin, Jiayi

- Provide an interface to control the arm
- Calibrate the inverse kinematics

Milestone 2 & 3

In these phases, the team can alternate implementing the vision part of the software while also making a chess board to play on. Together, we have to test if the arm is able to differentiate between pieces, locate them and move accordingly. Then, we implement the “thinking” with stockfish so that we can play against it.

Appendix

Summary of risks table

Risk	Impact	Likelihood	Mitigation / Early Prototype
Arm Rigidity and Flex	High	Medium	Reinforce the arm and forearm with rigidity pillars and full-length supports. Use lightweight but rigid materials, and keep the chess set and gripping system as light as possible.
Vertical Motion Stutter	Medium	Medium	Reduce friction on the three guide rails by adding smooth bushings (paliers lisses) to the MDF support. Test repeated up-and-down motion early in the prototyping phase.
Arm Reach and Positioning Limits	High	Medium	Validate the real reachable workspace with an early prototype. If some squares are difficult to reach precisely, adapt the chessboard size and placement accordingly.
ESP32-CAM Power Instability	Medium	Low	Keep the 5 V line short and thick, and add a low-ESR capacitor ($\geq 470 \mu\text{F}$) across the ESP32-CAM 5 V and GND pins. Test Wi-Fi and camera operation while motors are running.
Insufficient Current for Motors	High	Medium	Test how many motors can reliably operate at the same time. If necessary, limit the control strategy so that only one motor moves at a time.
Buck Converter Bottleneck	Medium	Medium	Monitor the circuit for current spikes and test the converter under sustained load. Add a heat sink if overheating becomes an issue.
ESP32-CAM Resolution / Image	High	Medium	Use strong visual contrast between the chessboard and the pieces. Test

Quality			different lighting conditions and camera positions, and use the onboard flash if necessary.
Low Color Saturation	High	High	Disable Auto White Balance (AWB) in the ESP32-CAM firmware, use more saturated board colors, and boost the saturation channel in OpenCV before HSV thresholding.
Camera Distance / Field of View	High	Medium	Improve framing through better camera placement or a zoom lens. Use area filtering to reject shapes that are too small or too large to correspond to valid pieces.
Image Capture Latency (Stale Frames)	Medium	High	Add a buffer-flushing routine so that the system always processes the newest available frame. Validate response time through repeated image capture tests.
Special Rule Cases	Medium	Medium	Handle castling as a dedicated software case, both when it is executed by the robot and when it is detected from the opponent's move. Validate it separately from standard moves.
Gripper force / slipping	High	Medium	Add high-friction pads, size the pinion and jaws with a safety factor, and test on the heaviest piece under realistic motion.